

Smart Hospital Patient & Resource Allocation System

Project Report



CSC 1012 Introduction to Computer Programming

Ms. S.A.M.P. Udeshika

Department of Computer Science

Student Name: P.S.R Wijesekara
Index : AS20240679

Date of Submission: 2026 September 20

Project :

Smart Hospital & Resource Allocation System

Course :

CSC 1012 — Introduction to Computer Programming

Assignment :

Individual Assignment

Language C

Data Structure Approach :

Parallel Arrays + 2D Bed Occupancy Array

Repository :

https://github.com/SenuraRansika/Smart_Hospital_and_Resource_Allocation_System

1. High-Level Architecture and Function List

The system follows a modular, menu-driven architecture implemented in C. The design separates the main user interaction loop from input validation, calculation functions, patient registration, priority sorting, bed monitoring, reporting and file persistence. This matches the assignment objective of applying arrays, custom functions, loops, conditionals, string handling, mathematical modelling and basic file handling

Menu and Program Control

Component	Current implementation
main()	Loads saved bed status, presents the main menu, validates the user's choice and dispatches the selected operation. On exit it saves the current bed matrix
Menu option 1	Register New Patient.
Menu option 2	View Bed Occupancy
Menu option 3	View Priority Queue (Sorted)
Menu option 4	Generate Reports
Menu option 5	Save bed status and exit

Core Workflow Controllers

Function	Purpose
registerPatient()	Handles complete patient intake, specialty and triage selection, admission decision, bed allocation, bill calculation, queue update and billing-log append
showPriorityQueue()	Displays registered patients in urgency priority using a temporary index array and bubble sort.
displayBeds()	Displays the available/occupied state of each usable bed and calculates ward occupancy percentage.
generateReports()	Produces patient totals by urgency, total revenue, total discounts, occupancy by ward and the highest-paying patient.

Utility and Mathematical Functions

Function	Purpose
getValidInt()	Validates integer input against a supplied range and clears invalid input from the buffer.
getValidName()	Reads a non-empty patient name containing spaces.
calculateWaitTime()	Current specialty queue count × consultation time.
calculateSurcharge()	Level 1 = 0%; Level 2 = 20%; Level 3 = 50% of the specialty base fee.
calculateWardCost()	Days admitted × selected ward daily rate; zero when not admitted.
calculateDiscount()	15% of gross bill when age < 5 or age > 65; otherwise zero.

Utility and Mathematical Functions

Function	Purpose
saveBedsToFile()	Writes the bedOccupancy matrix to beds_status.txt.
loadBedsFromFile()	Reads the saved bedOccupancy matrix at application startup.
appendPatientRecord()	Appends patient ID, name, age, urgency level and final bill to patient_records.txt.

System Processing Model

Patient Intake and Resource Allocation

THE REGISTRATION PROCESS FIRST VALIDATES THE MAXIMUM PATIENT CAPACITY, THEN COLLECTS NAME, AGE, TRIAGE LEVEL AND SPECIALTY. IF ADMISSION IS REQUESTED, THE PROGRAM COLLECTS THE WARD AND NUMBER OF DAYS AND SCANS THE SELECTED WARD FROM ITS FIRST USABLE BED UNTIL IT FINDS A VALUE OF 0. THE SELECTED BED IS CHANGED TO 1. IF NO BED IS AVAILABLE, THE CODE CANCELS THE ADMISSION FOR THAT RECORD AND SETS THE ADMISSION-RELATED VALUES TO THEIR NON-ADMITTED STATE.

Billing Model

GROSS TOTAL = BASE CONSULTATION FEE + EMERGENCY SURCHARGE + WARD STAY COST

EMERGENCY SURCHARGE :

LEVEL 1 = 0;

LEVEL 2 = BASE FEE × 0.20;

LEVEL 3 = BASE FEE × 0.50

WARD STAY COST = DAYS ADMITTED × WARD DAILY RATE, OR
0 FOR OUTPATIENTS

AGE DISCOUNT = GROSS TOTAL × 0.15 WHEN AGE < 5 OR AGE > 65

FINAL PAYABLE = GROSS TOTAL – AGE DISCOUNT

Priority Queue

showPriorityQueue() creates indexArr[MAX_PATIENTS], fills it with patient indices, and bubble-sorts the indices according to triageLevel in descending order. The patient arrays themselves are not rearranged. Because equal triage levels are not swapped, patients with the same urgency retain their registration order, satisfying the assignment's secondary priority rule.

Lookup Data

ID	Specialty	Base Fee (LKR)	Time / Patient	Daily Cap
1	General Practice (OPD)	1,500.00	15 mins	30
2	Paediatrics	2,500.00	20 mins	20
3	Cardiology	4,500.00	30 mins	12
4	Neurology	5,000.00	30 mins	10

Ward ID	Ward	Daily Rate (LKR)	Capacity
1	General Ward	3,000.00	20
2	Paediatric Ward	6,000.00	10
3	Surgical Ward	12,000.00	10
4	ICU	25,000.00	5

Note: dailyCap[] is present as required lookup data, but the current registration logic does not enforce the specialty daily-cap values.

2. Data Structure Design

The current implementation uses that permitted parallel-array approach for patient data and a 2D integer matrix for beds.

Patient Parallel Arrays

Array	Declaration	Purpose
patientName	char[100] [50]	Stores up to 100 patient names.
patientAge	int[100]	Stores patient age.
triageLevel	int[100]	Stores urgency: 1 Normal, 2 Urgent, 3 Critical.
patientSpecialty	int[100]	Stores specialty index 0–3.
isAdmitted	int[100]	Stores admission status (1/0).
patientWard	int[100]	Stores ward index; -1 for outpatient.
daysAdmitted	int[100]	Stores number of admitted days.
assignedBedNo	int[100]	Stores zero-based assigned bed index; -1 if not assigned.
finalBill	float[100]	Stores final payable amount.
queueCount	int[4]	Tracks registered patients per specialty.

Lookup and Resource Arrays

Array	Declaration	Purpose
specialtyID	int[4]	Specialty IDs.
specialtyName	char[4][30]	Specialty names.
baseFee	float[4]	Consultation base fees.
consultTime	int[4]	Consultation time per patient.
dailyCap	int[4]	Configured specialty daily capacity.
wardID	int[4]	Ward IDs.
wardName	char[4][30]	Ward names.
wardDailyRate	float[4]	Daily ward rates.
wardCapacity	int[4]	Usable ward capacities.

2D Bed Occupancy Matrix

int bedOccupancy[NUM_WARDS][MAX_BEDS_PER_WARD] is a 4 × 20 matrix. Each position stores 0 for Available and 1 for Occupied. The actual number of positions considered for allocation is controlled by wardCapacity[]. Internally bed numbers are zero-based; the displayed bill adds 1 so the user sees Bed #01, Bed #02, etc.

Temporary Priority Index

`int indexArr[MAX_PATIENTS]` is used only while displaying the priority queue. This avoids moving the actual patient records and keeps the parallel arrays synchronized.

3. GitHub Repository and File Handling

GitHub Repository

https://github.com/SenuraRansika/Smart_Hospital_and_Resource_Allocation_System

The repository is the project source-control location supplied for this submission. The assignment requires a public GitHub repository with at least 15 meaningful commits distributed across the project timeline.

File Handling

File	Mode	Purpose
beds_status.txt	w / r	Stores and restores the 4 × 20 bed occupancy matrix across program runs
patient_record s.txt	a	Maintains an append-only billing log containing patient ID, name, age, urgency and final bil

The current implementation loads bed occupancy when the program starts and saves it when the user exits. Patient billing records are appended immediately after registration. The current program does not reload those historical patient records into the parallel arrays, so current-session reports and priority queues operate only on patients registered during the current execution.

Performance Reports and Analytics

Required Report	Current implementation
Patients by urgency	Counts Normal, Urgent and Critical patients.
Total revenue	Sums finalBill[] for the current in-memory patient records.
Total discounts	Reconstructs gross billing and sums gross – finalBill.
Ward occupancy	Counts occupied beds and divides by configured ward capacity
Highest-paying patient	Finds the maximum finalBill and displays the patient's name and amount.

4. Assumptions Made During Development

- The hospital contains four predefined specialties and four predefined wards.
- The usable ward capacities are 20, 10, 10 and 5 beds, while the physical matrix has 20 positions for every ward.
- Bed status is represented by 0 = available and 1 = occupied.
- The system supports a maximum of 100 patient records in memory during one program execution.
- Patient IDs are generated from registration order using PAT-1001 for the first patient.
- Triage levels are Level 1 = Normal, Level 2 = Urgent and Level 3 = Critical.
- The first available bed in the selected ward is assigned automatically.
- If the selected ward has no available usable bed, the patient remains registered but is changed to outpatient/non-admitted status.
- Bed numbers are stored internally as zero-based indices and displayed to users as one-based bed labels.
- Waiting time uses the current specialty queue count before the new patient's queue count is incremented.
- Urgency surcharges follow the assignment values of 0%, 20% and 50%.
- Patients younger than 5 or older than 65 receive a 15% discount on gross billing.

- Outpatients have zero ward cost.
- The `dailyCap[]` values are stored as lookup data but are not enforced by the current registration logic.
- Bed occupancy persists between executions through `beds_status.txt`.
- `patient_records.txt` is treated as an append-only audit/billing log and is not reloaded into active patient arrays.
- Revenue in `generateReports()` is defined as net collected income, represented by the sum of `finalBill[]`.
- Specialty queue counts reset when the program starts because `queueCount[]` is initialized to zero.

Reference-Report Integration Note

The additional five-page reference report supplied for this project uses a similar report organization: high-level architecture, function list, data structures, file persistence and system assumptions. Its discussion of structs was not copied into this report because the current source code uses parallel arrays. Likewise, reference-only functions such as `resetSystem()`, `displayMenu()`, `assignBed()`, `calculateGrossTotal()`, `calculateFinalAmount()`, `formatCurrency()`, `printBill()`, `saveData()/loadData()` and `savePatientRecord()` are not listed as implemented functions here because they do not exist in the supplied current source.

5. Future Enhancements & Conclusion

Future Enhancements

Dynamic Memory Allocation :

Transitioning from fixed-size parallel arrays to linked lists (malloc/free) to allow unlimited patient capacity. Graphical User Interface (GUI): Upgrading from a console interface to a desktop GUI (e.g., using GTK or Win32 API) for improved usability. Database Integration: Replacing text file storage (.txt) with an SQL database (e.g., SQLite) for structured querying and data encryption. Role-Based Access Control: Implementing multi-user login mechanisms for Doctors, Administrative Staff, and Pharmacists with restricted access levels

Conclusion

The Smart Hospital & Resource Allocation System implements the core CSC 1012 requirements through a modular C program. It combines patient intake, emergency triage, specialty waiting-time estimation, automatic bed allocation, billing, priority sorting, performance reporting and text-file persistence in a single console-based workflow.

The data model uses parallel arrays for patient records and a 4×20 integer matrix for bed status, which is consistent with the assignment's permitted approach. The system also implements the specified billing equations and report calculations.

The main documented limitations are the fixed 100-patient in-memory capacity, current-session-only patient analytics, append-only historical billing records, and the absence of enforcement for the configured specialty daily-cap values. These limitations are stated explicitly so that the report represents the current source code accurately.